Requirements Engineering

CSE 362

Software Development and Professional Practice

Winter 2024

© Royal Military College of Canada

Requirements engineering, is the first major phase in a software development project.

The most challenging and difficult aspect of requirements engineering is to get a complete description of the problem to be solved.

We discuss some techniques for eliciting requirements from the user. Following elicitation, these requirements must be negotiated, validated, and documented.

- A requirement is ‘a condition or capability needed by a user to solve a problem or achieve an objective’

- The word ‘requirement’ suggests that, once stated, it **has** to be met. In reality, this hardly ever is the case. Most requirements are negotiable…

- The result of the requirements engineering phase is documented in the **requirements specification**: basis for a contract, be it formal or informal, between the client of the system and the developers.

During requirements engineering, a number of quite different matters are being addressed.

Let us look at an example and consider the (hypothetical) case of a university's library automating its operation.

A university's library: current situation

- The library contains a number of cabinets.
- These cabinets hold a huge number of cards, one per book.
- Each card contains the *names of the authors*, the *book title*, *ISBN*, *publication year*, and other useful data.
- The cards are *ordered alphabetically* by the name of the first author of each book

Current problems with current system

- It only works well if we know the first author's name.
  - If we only know the title, or if we are interested in books on a certain topic, the author catalog is of little or no help.

- A software solution seems obvious.

- If we store the data for each book once in a database, we may subsequently sort the entries in many different ways.

A university's library: Requirements

- Reqs related to updating the database:
  - adding, deleting and changing records.

- Others reqs will concern functions to be provided to ordinary members of the library, such as:
  - Give a list of all books written by X
  - Give a list of all books whose title contains Y;
  - Give a list of all books on topic Z;
  - Give a list of all books that arrived after date D.

A university's library: Requirements

- Try to somehow group user requirements into a few categories, ranging from 'essential requirements' to 'nice features'.

- Example: '*Give a list of all books that arrived after date D*' could be classified as a 'nice feature'.
  - Service is not seriously degraded if this function is not provided.

A university's library: Requirements

- Also try to predict a number of future requirements, to be implemented in future releases?

- Why? To make sure they could be accommodated during the design of the system.

- Examples:
  - Storing information about books that have been ordered but have not been received;
  - Storing information about library members, such as their name and address, and the dates on which books are lent to them, which can then be used to generate a reminder notice for books not returned on time.

A university's library: Requirements

- Think about all types of stakeholders.

- For example, library management may wish to use the system to get information on member profiles in order to improve the title acquisition process.

A university's library: Requirements

- Besides the previous requirements, i.e. functions that directly relate to the software to be delivered (functional requirements), other *type of matters* (Non-Functional Requirements) should be considered.

- In this example, at a minimum, the following should be addressed:
  - On which machine will the system be implemented, and which operating system will be used? If the data is to be stored in some DBMS, which (type of) DBMS is to be used?
  - Which classes of users can be distinguished? In our example, both library personnel and library members will have to be served. Will certain functions of the system be restricted to certain classes of users.

A university's library: Requirements

- What is the size of the database and how is it expected to grow in the course of time? These factors influence both storage capacity needed and algorithms to be used.

- What response time should the system offer? A search request for a certain book will have to be answered fairly quickly. If the user has to wait too long for an answer, he will become dissatisfied and search the shelves directly.

- How much will a system of this kind cost?

Takeaway

This relatively simple example already shows that it is not sufficient to merely list the functional requirements of the new system. The system's envisioned **environment** and its **interaction** with that environment should be analyzed as well.

Four processes in requirements engineering

- **Requirements elicitation:** understanding the problem.

- **Requirements specification:** describing the problem.

- **Requirements validation and verification:** Once the problem is described, the different parties involved have to *agree upon* its nature. the correct requirements are stated (validation) and that these requirements are stated correctly (verification)

- **Requirements negotiation:** solving conflicts between stakeholders.

Requirements Elicitation Techniques

- The two main sources of information for the requirements elicitation process are the users and the (application) domain.

- You should generally vacuum a rug in two directions rather than one.
  - You should use multiple requirements elicitation techniques

Requirements Elicitation Techniques

- **Asking:** We may simply ask the users what they expect from the system. A presupposition then is that the user is able to bypass their own limitations and prejudices.
  - Asking may take the form of an interview, a brainstorm, or a questionnaire.

- **Task analysis:** a technique to obtain a hierarchy of tasks and subtasks to be carried out by people working in the domain.
  - Employees working in some domain perform a number of tasks, such as handling requests to borrow a book, cataloging new books, ordering books, etc. Higher-level tasks may be decomposed into subtasks.

Requirements Elicitation Techniques

- **Scenario-based analysis:** Instead of looking for generic plans as in interviews or task analysis, the analyst may study **instances** of tasks. A scenario is a story which tells us how a specific task instance is executed. The scenario can be real or artificial.

- **Ethnography**: Groups of people are studied in their natural settings.

- **Form analysis** A lot of information about the domain being modeled can often be found in various forms being used.

Requirements Elicitation Techniques

- **Natural language descriptions** Like forms, natural language descriptions provide a lot of useful information about the domain to be modeled (e.g. operating instructions for library employees).

- **Derivation from an existing system**

- **And more**

Prioritizing Requirements

- **Must haves**: these are the top priority requirements.
- **Should haves**: these requirements are not strictly mandatory, but they are highly desirable.
- **Could haves**: if time allows, these requirements will be realized as well.
- **Won't haves**: these requirements will not be realized in the present version. They are recorded though.

Before you start...

- You should have agreed on a project theme and have your theme approved by the prof.

- Normally Requirements are obtained from your client - but for this course, you are your client!

- So, pretend: If you were buying this system, what would you expect it to do?

Requirements

- Functional Requirements are what your system does. You might be calling these things "features" or "goals".

- Non-Functional Requirements are about how well your system does what it does. These form the boundaries around your functional requirements. Things like processing time restrictions, OS and platform restrictions, memory usage restrictions, number of concurrent users, maintenance, etc.

Requirements

- So as you are listing functional requirements, you must always consider the restrictions on those requirements.

- It is important to separate requirements along these boundaries because the requirements will be treated differently:

– Functional Requirements result in <u>tasks that must be completed</u>. Testing will verify that the task is complete.

– Non-Functional Requirements <u>result in tests directly</u>. Tasks will be created to address the problem of a non- functional requirement that is not met.

How to come up with requirements in your case?

- Try to think through every aspect of your system operation.
- Sketch out GUI prototypes.
- Some students like to use flowcharts to describe how the system operates. (Using proper UML diagrams is not necessary at this point. But, note there are drawing systems such as draw.io...)

How to come up with requirements in your case?

- Every requirement must be verifiable. Can you create a test that verifies that the requirement has been met?

- Everyone has to agree on what a requirement means - this often is a challenge for the system architect to make sure he has clearly communicated his vision of the system.

- Communicate!!

How to come up with requirements in your case?

- Don't worry <u>yet</u> about how long a requirement will take to implement or who is going to do it.

- Requirements are not just about writing software, you may require other tasks such as research, algorithm choice and testing, art, writing a backstory, creating a help document, etc.

- Later, you will consider the sequence of tasks, as well.

Documenting Requirements

- Everyone on the team needs to see and agree on the requirements.

- We will use the following technique to list these requirements:

- The RAD - "Requirements Analysis Document".
- It is a bit old-fashioned, but it is easy enough to create and will force you to think through your system.
- It is graded.

Project Deliverable - the RAD

- "Requirements Analysis Document".
- The purpose of this deliverable is to make sure you have started on the design of your app.
- You don't need to have written any code, but after you have created the RAD, you should have a pretty good overall idea of what you are building.
- You still may not have decided on the architectural details (to be done in the design phase!).

Project Deliverable – the RAD, Contents

- Broad-view summary of what you are planning to build. Consider inputs and outputs and what you are trying to accomplish. Who are the target user(s)? A diagram might best describe these concepts.
- A list of functional and non-functional requirements expressed as user stories. You may have been calling these "features". What exactly does the program have to do, and in what order?
- Sketches or prototypes of the GUI interface (all windows) along with a description of how the user will navigate through the interface.
- A list of the other roles assigned to each team member, on the assumption that everyone is a coder first.

Project Deliverable – the RAD, Outline

1. Project Overview (**out of 4**)
   - Background/history
   - Purpose
   - Scope
   - Target users
   - Objectives and success criteria
2. System Description (**out of 5**)
   - Definitions and Acronyms
   - Layout Description
   - Parameters
   - Input/Outputs
   - Assumptions/Constraints

Project Deliverable – the RAD, Outline

- Requirements (**out of 10**)
  - Functional
  - Non-functional
- Prototypes of GUI Interface (**out of 5**)
- Team Name and Roles (**out of 2**)

- Total: 26 points